به نام خدا



دانشکده ریاضی و آمار

اصول سیستم‌های عامل
نیم‌سال اول ۰۴-۰۵
استاد: دکتر رفیعی
نام دانشجو - شماره دانشجویی

تمرین سری اول

۱. (آ) توزیع‌کننده یکی از مولفه‌های ... است. این مولفه در واقع یک ... است که کنترل ... را در اختیار فرآیند انتخاب شده توسط زمان‌بند CPU قرار می‌دهد.

(ب) دو مورد از وظایف عملکردی توزیع‌کننده را نام ببرید.

(ج) آیا سرعت این مولفه بر روی عملکرد کلی سیستم‌عامل تاثیر می‌گذارد؟ چرا؟

پاسخ:

(آ) زمان‌بندی CPU، ماژول، هسته‌ی پردازشی

(ب) دو مورد از سه مورد زیر به دلخواه:

۱. تعویض محتوای یک فرآیند با یک فرآیند دیگر (Context Switching)
۲. تعویض به مد کاربر
۳. پرش به مکان مناسب از برنامه‌ی کاربر برای ادامه‌ی اجرای برنامه

(ج) بله. چرا که در هر تعویض متن فراخوانی می‌شود.

۲. (آ) چرا مدت زمان انفجارهای CPU در سیستم اندازه‌گیری می‌شود؟

(ب) آیا صف فرآیندهای آماده (Ready Queue) لزوماً باید به صورت FIFO پیاده‌سازی شود؟

(ج) دو مورد از ساختارهای داده‌ای که می‌توان برای پیاده‌سازی صف آماده استفاده کرد را نام ببرید.

پاسخ:

(آ) جهت استفاده در پیاده‌سازی الگوریتم‌های زمان‌بندی CPU.

(ب) خیر.

(ج) صف اولویت، درخت، لیست پیوندی نامرتب (دو مورد ذکر شود).

۳. (آ) اگر زمان‌بندی فقط در شرایط «تغییر از اجرا به انتظار» و «خاتمه فرآیند» صورت گیرد، به آن طرح زمان‌بندی ... می‌گوییم.

(ب) اکثر سیستم‌عامل‌های مدرن از چه مکانیزمی برای زمان‌بندی فرآیندهای کرنل استفاده می‌کنند؟

(ج) در اسلایدها خواندیم که بخش‌هایی از کد وجود دارند که نمی‌توانند توسط چند فرآیند به صورت همروند استفاده شوند. از این رو ... ها هنگام ورود به این بخش غیرفعال و به هنگام خروج فعال می‌شوند.

پاسخ:

(آ) اگر زمان‌بندی فقط در شرایط «تغییر از اجرا به انتظار» و «خاتمه فرآیند» صورت گیرد، به آن طرح زمان‌بندی ... می‌گوییم.

(ب) زمان‌بندی قبضه‌ای.

(ج) وقفه‌ها

۴. در این سوال قصد داریم تا دو مشکل مربوط به **اجرای قبضه‌ای** فرآیندها را مورد بررسی قرار دهیم (به صفحه ۱۷ اسلاید ۱۳ مراجعه شود).

(آ) (**مشکل ۱: از دست رفتن ورودی**) فرض کنید که دو فرایند داریم. فرایند ۱ دائم اطلاعات مربوط به I/O را از یک دستگاه دریافت می‌کند و آن را در یک بافر مشترک قرار می‌دهد. فرایند ۲ نیز دائم داده‌ها را از این بافر می‌خواند و پردازش می‌کند. در صورت اجرای قبضه‌ای این دو فرایند، چه مشکلی ممکن است رخ دهد؟ لطفاً با استفاده از کد زیر که نمایانگر هر یک از این دو فرایند است، مشکل را توضیح دهید.

دقت داشته باشید که متغیر buffer بین هر دو فرآیند مشترک است.

متغیر new_data نیز داده جدیدی است که از دستگاه I/O دریافت شده است.

```c
while (1) {
    // Producer (Interrupt
       )
    buffer = new_data;
}
```

فرآیند ۱

```c
void kernel_process() {
    // Consumer
    int data = buffer;
    process(data);
}
```

فرآیند ۲

(ب) **(مشکل ۲: بازنویسی خروجی)** فرض کنید که یک تابع به نام increment داریم که یک شمارنده مشترک (counter) را افزایش می‌دهد. پیاده‌سازی این تابع را در زیر می‌بینید. این پیاده‌سازی تقریبا معادل به اتفاقاتی است که هنگام اجرای counter++ در زبان C رخ می‌دهد. حالا فرض کنید این تابع دوبار توسط تابع main صدا زده شود. چه مشکلی ممکن است رخ دهد؟ توضیح دهید.

```c
void increment() {
    int temp;
    temp = counter;
    temp = temp + 1;
    counter = temp;
}
```

تابع افزایش مقدار شمارنده

```c
int main() {
    // Shared counter = 5
    increment();
    increment();
}
```

تابع main

پاسخ:

(آ) این توالی را در نظر بگیرید:

۱. فرآیند ۱ داده جدیدی را در بافر می‌نویسد.

۲. فرآیند ۱ توسط سیستم‌عامل متوقف می‌شود (قبضه‌ای).

۳. فرآیند ۱ دوباره اجرا می‌شود و داده جدید دیگری را در بافر می‌نویسد (بدون اینکه به فرآیند ۲ فرصت بدهد تا داده قبلی را بخواند).

۴. فرآیند ۲ اجرا می‌شود و داده‌ای را از بافر می‌خواند.

در این سناریو، داده‌ای که فرآیند ۲ می‌خواند، داده‌ای است که توسط فرآیند ۱ در مرحله ۳ نوشته شده است. بنابراین، داده‌ای که در مرحله ۱ نوشته شده بود، از دست رفته است زیرا فرآیند ۲ هرگز فرصتی برای خواندن آن نداشت. این مشکل به دلیل اجرای قبضه‌ای رخ می‌دهد که اجازه نمی‌دهد فرآیند ۲ قبل از اینکه فرآیند ۱ دوباره بافر را بازنویسی کند، داده را بخواند.

(ب) این توالی را در نظر بگیرید:

۱. تابع main اولین بار تابع increment را صدا می‌زند.

۲. درون ،increment مقدار شمارنده مشترک (۵) در temp خوانده می‌شود.
۳. قبل از اینکه temp افزایش یابد و مقدار جدید در شمارنده نوشته شود، سیستم‌عامل اجرای تابع increment را متوقف می‌کند (قبضه‌ای).
۴. تابع main دومین بار تابع increment را صدا می‌زند.
۵. درون ،increment مقدار شمارنده مشترک (۵) دوباره در temp خوانده می‌شود.
۶. temp افزایش می‌یابد و مقدار جدید (۶) در شمارنده نوشته می‌شود.
۷. سپس سیستم‌عامل اجرای اولین فراخوانی increment را از سر می‌گیرد.
۸. temp که هنوز مقدار ۵ را دارد، افزایش می‌یابد و مقدار جدید (۶) در شمارنده نوشته می‌شود.

در این سناریو، انتظار داشتیم که پس از دو فراخوانی ،increment مقدار شمارنده به ۷ افزایش یابد. اما به دلیل اجرای قبضه‌ای، هر دو فراخوانی مقدار اولیه ۵ را خواندند و در نهایت شمارنده فقط به ۶ افزایش یافت. این مشکل به دلیل عدم هماهنگی بین دو فراخوانی رخ داد که منجر به بازنویسی خروجی شد.

۵. سه فرآیند P3, P2, P1 با زمان اجرا و زمان ورود زیر را در نظر بگیرید و به موارد خواسته شده پاسخ دهید. (quantum = 2)

فرآیند	زمان ورود	زمان اجرا
$P1$	0	6
$P2$	1	3
$P3$	6	4
$P4$	6	2

۱. بهره وری CPU

۲. میانگین زمان برگشت

۳. میانگین زمان انتظار

۴. میانگین زمان پاسخ

۶. (آ) تفاوت دیدگاه کاربر و دیدگاه سیستم در ارزیابی الگوریتم‌های زمان‌بندی چیست؟

پاسخ:
از دیدگاه کاربر، هدف اصلی کاهش زمان پاسخ (Response Time) است تا سیستم سریع‌تر واکنش نشان دهد. اما از دیدگاه سیستم، هدف اصلی افزایش توان عملیاتی (Throughput) و به حداکثر رساندن بهره‌وری پردازنده (CPU Utilization) است تا از منابع سخت‌افزاری به بهترین شکل استفاده شود.

(ب) منظور از زمان بازگشت (Turnaround Time) چیست و چگونه محاسبه می‌شود؟

پاسخ:
زمان بازگشت بیانگر کل زمان سپری شده از لحظه ورود فرآیند به صف آماده تا لحظه تکمیل و خروج آن از سیستم است. این زمان را می‌توان با فرمول «زمان خروج منهای زمان ورود» یا «مجموع زمان اجرا و زمان انتظار» محاسبه کرد.

۷. (آ) اگر در الگوریتم زمان‌بندی نوبتی-گردشی ،(RR) **کوانتوم زمانی** بزرگ‌تر از زمان اجرای طولانی‌ترین فرآیند موجود در سیستم باشد، این الگوریتم به چه سیاستی تبدیل می‌شود؟

پاسخ:
در این شرایط، هر فرآیند پیش از اتمام کوانتوم زمانی، اجرای خود را کامل کرده و داوطلبانه پردازنده را رها می‌کند؛ بنابراین سیاست RR به الگوریتم **FCFS** (سرویس به ترتیب ورود) تنزل می‌یابد [۱۲].

(ب) برای جلوگیری از پدیده **گرسنگی** در زمان‌بندی اولویت‌دار، از چه مکانیزمی استفاده می‌شود؟

پاسخ:
از الگوریتم **سالمندی (Aging)** استفاده می‌شود. در این روش، اولویت فرآیندهایی که برای مدت طولانی در سیستم منتظر مانده‌اند، به طور تدریج افزایش می‌یابد (به عنوان مثال، با کاهش عدد اولویت در سیستم‌هایی که عدد کمتر نشان‌دهنده اولویت بالاتر است) تا در نهایت شانس اجرا پیدا کنند [۴۸، ۴۹].

(ج) چرا الگوریتم **SJF** از نظر تئوری یک الگوریتم **بهینه** محسوب می‌شود؟

پاسخ:
این الگوریتم بهینه است زیرا برای یک مجموعه مشخص از فرآیندها، **کمینه (حداقل) میانگین زمان انتظار** را فراهم می‌کند [۲۴]. با جابه‌جا کردن یک کار کوتاه به قبل از یک کار بلند، زمان انتظار فرآیند کوتاه بیشتر از زمان انتظار فرآیند بلند کاهش می‌یابد [۲۴، ۲۵].

(د) کوچک انتخاب کردن بیش از حد **کوانتوم زمانی** در الگوریتم RR چه پیامدی برای سیستم دارد؟

پاسخ:
باعث افزایش تعداد دفعات **تعویض متن (Context Switch)** می‌شود. از آنجا که هر تعویض متن سربار زمانی به سیستم تحمیل می‌کند، کوچک بودن بیش از حد کوانتوم منجر به کاهش شدید **توان عملیاتی (Throughput)** سیستم می‌گردد [۱۳].

۸. (آ) با فرض استفاده از فرمول $S_n = \alpha T_{n-1} + (1-\alpha)S_{n-1}$ برای پیش‌بینی انفجار بعدی در **SJF**، اگر $\alpha = 01$ تنظیم شود و یک فرآیند پس از مدت‌ها اجرای کوتاه، ناگهان یک انفجار بسیار طولانی داشته باشد، واکنش سیستم را تحلیل کنید.

پاسخ:
از آنجا که مقدار α کوچک است (۰۱)، وزن داده شده به آخرین رخداد (T_{n-1}) بسیار ناچیز است و سیستم بیشتر بر اساس **تاریخچه پیشین** (S_{n-1}) تصمیم می‌گیرد [۳۱، ۳۲]. در نتیجه، سیستم به تغییر رفتار ناگهانی فرآیند با کندی واکنش نشان می‌دهد و همچنان آن را فرآیندی کوتاه در نظر می‌گیرد تا زمانی که چندین بار پیاپی انفجارهای طولانی تکرار شوند [۳۰، ۳۳].

(ب) تحلیل کنید که چرا در الگوریتم **MLFQ**، فرآیندهای Bound I/O در صفوف بالا باقی می‌مانند اما فرآیندهای Bound CPU به صفوف پایین رانده می‌شوند؟

پاسخ:
در MLFQ، اگر فرآیندی تمام کوانتوم زمانی خود را مصرف کند (ویژگی فرآیندهای CPU-Bound)، سیستم اولویت آن را کاهش داده و به صف پایین‌تر منتقل می‌کند [۶۰، ۶۱]. اما فرآیندهای Bound I/O معمولاً پیش از اتمام کوانتوم، پردازنده را برای انجام عملیات ورودی/خروجی رها می‌کنند و به همین دلیل در صفوف با **اولویت بالاتر** باقی می‌مانند که این امر منجر به پاسخ‌دهی سریع‌تر به کارهای تعاملی می‌شود [۶۱].